

FEDERATED LEARNING FOR CODE GENERATION WITH HALLUCINATION-AWARE FINE TUNING

A PROJECT REPORT

Submitted by

ABHINAVH P

PRABHAKARA ARJUN R

BUVANES SRIVARDAN K

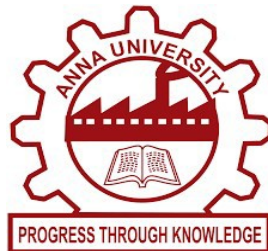
in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER TECHNOLOGY

MADRAS INSTITUTE OF TECHNOLOGY CAMPUS

ANNA UNIVERSITY, CHENNAI 600 044

APRIL 2026

BONAFIDE CERTIFICATE

Certified that the bonafide thesis report of the project work, **“FEDERATED LEARNING FOR CODE GENERATION WITH HALLUCINATION-AWARE FINE TUNING”** done by **Mr. Abhinavh P (2022503059), Mr. Prabhakara Arjun R(2022503003) & Mr. Buvanes Srivardan K (2022503037)** who carried out the project work under my supervision.

SIGNATURE OF SUPERVISOR

Dr. THANASEKHAR B

Professor

Department of Computer Technology

Anna University, MIT Campus

Chromepet, Chennai-600044

SIGNATURE OF HOD

Dr. PONSY R K SATHIA BHAMA

Head of Department

Department of Computer Technology

Anna University, MIT Campus

Chromepet, Chennai-600044

ABSTRACT

Large Language Models (LLMs) have demonstrated strong capabilities in automated code generation, enabling solutions for a wide range of software engineering tasks. However, these models often suffer from hallucinations, producing syntactically correct but semantically incorrect code, including invalid APIs and logical errors. Such issues reduce the reliability of LLM based systems in practical applications.

In this project, a federated learning based framework for improving code generation by reducing hallucinations in a privacy preserving manner is proposed. The system integrates hallucination detection to identify and correct errors in generated code, thereby creating high quality datasets for model adaptation. These datasets are used to fine tune the model using parameter-efficient Low Rank Adaptation (LoRA) across multiple decentralized clients.

Each client performs local fine tuning using its own data and only lightweight adapter parameters are shared with a central server, ensuring data privacy. Unlike traditional federated learning approaches that use standard Federated Averaging (FedAvg), this work introduces Federated Error Average (FedErrorAvg), where updates are weighted based on dataset size and hallucination error distribution across clients. This error-aware aggregation enables the model to learn more effectively from diverse error patterns, improving generalization and reducing hallucinations.

Experimental results demonstrate progressive improvement: total hallucinations reduce from 1,791 (baseline) to 1,635 (FedAvg) to 1,582 (FedErrorAvg), an 11.7% reduction. Pass rate improves from 52.4% to 61.2% to 62.6%. NameError drops 84%, SyntaxError 56% and TypeError 70% under FedErrorAvg, significantly outperforming FedAvg.

ACKNOWLEDGEMENT

We take this opportunity to sincerely thank **Prof. Dr. Jayashree P**, respected Dean, Anna University, MIT Campus, and **Dr. Ponsy R K Sathia Bhama**, respected Head of the Department, Department of Computer Technology, for providing the essential facilities that enabled us to pursue this project.

We would like to thank our supervisor **Dr. B.Thanasekhar**, Professor, Department of Computer Technology, Anna University, MIT Campus, whose kind guidance and proper directions helped shape this project perfectly. Constant encouragement to implement the project to the fullest of our potential propelled us toward the successful completion of the proposed project work.

We acknowledge the efforts and feedback of the panel members, **Dr. P. Pabitha**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr. S. Chithra**, Associate Professor, Department of Computer Technology, MIT, Anna University and **Dr. V.P. Jayachitra**, Assistant Professor, Department of Computer Technology, MIT, Anna University in reviewing our work, providing constant valuable comments and encouraging us to view the different aspects of the project in successful implementation of the project.

We thank all the teaching and non-teaching members of the Department of Computer Technology, MIT, Anna University and to my family and friends whose appreciable help, either directly or indirectly helped in the smooth completion of the Project work within the stipulated timeframe.

ABHINAVH P (2022503059)

PRABHAKARA ARJUN R (2022503003)

BUVANES SRIVARDAN K (2022503037)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	1
	1.1 LARGE LANGUAGE MODELS IN CODE GENERATION	1
	1.2 HALLUCINATIONS IN CODE GENERATION	2
	1.3 FEDERATED LEARNING	3
	1.4 PARAMETER EFFICIENT FINE TUNING	3
	1.5 PROBLEM STATEMENT	4
	1.6 OBJECTIVE	5
	1.7 ORGANIZATION OF THE REPORT	6
2	LITERATURE SURVEY	8
	2.1 CODE GENERATION AND RELIABILITY OF LLM OUTPUTS	8
	2.2 HALLUCINATIONS DETECTION AND MITIGATION TECHNIQUES	10
	2.3 EFFICIENT FINE-TUNING OF CODE GENERATION MODULE	12

	2.4 FEDERATED LEARNING AND AGGREGATION STRATEGIES	13
	2.5 SUMMARY OF LITERATURE	15
3	HALLUCINATION-AWARE FEDERATED FRAMEWORK FOR LLM	18
	3.1 SYSTEM OVERVIEW	18
	3.2 OVERALL ARCHITECTURE	19
	3.2.1 Client-Side Hallucination Detection and Dataset Construction	19
	3.2.2 LoRA Fine-Tuning and Federated Aggregation	20
	3.3 MODULES BREAKDOWN	22
	3.3.1 Code Generation Module	22
	3.3.2 Hallucination Detection Module	24
	3.3.3 Dataset Creation	26
	3.3.4 LoRA Fine-Tuning module	28
	3.3.5 Federated Aggregation Module	29
	3.3.6 Verification Module	31
4	IMPLEMENTATION	33
	4.1 RUNTIME ENVIRONMENT	33
	4.1.1 Google Colab	33
	4.1.2 Visual Studio and Git	33
	4.1.3 JupyterHub	34
	4.2 LIBRARIES USED	34
	4.3 DATASETS USED	35
	4.4 LLM USED	37
	4.5 MODULE WISE IMPLEMENTATION	38

4.5.1	Code Generation Module	38
4.5.2	Hallucination Detection Module	39
4.5.3	LoRA Fine-Tuning	41
4.5.3.1	LoRA Adaptor Count Calculation	43
4.5.3.2	Reduction in Trainable Parameters	43
4.5.4	Federated Aggregation Module	44
4.5.5	Verification Module	47
5	RESULTS AND ANALYSIS	48
5.1	EVALUATION METRICS	48
5.1.1	Pass Count	48
5.1.2	Error Count	48
5.1.3	CodeBLEU	49
5.1.4	Error Reduction	49
5.1.5	Pass Rate	50
5.2	HALLUCINATION DETECTION ANALYSIS	50
5.3	TYPES OF HALLUCINAIONS ACROSS DATASET	51
5.4	FEDERATED AVERAGE AND BASELINE	53
5.5	FEDERATED ERROR AVERAGE AND BASELINE	55
5.6	PASSRATE COMAPRISON	56
5.7	ERROR REDUCTION ANALYSIS	58
5.8	CODEBLEU SIMILARITY	61

6	CONCLUSION AND FUTURE ENHANCEMENTS	63
	REFERENCE	66

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
4.1	Libraries Used	34
4.2	Dataset Summary	36
4.3	HumanEval Summary	36
4.4	MBPP Summary	37
4.5	DS1000 Summary	37
4.6	Qwen-coder Configuration	38
4.7	LoRA Configuration	42
4.8	SFT Configuration	42
4.9	Reduction Count in Trainable parameters	43
4.10	Client Sample sizes and Aggregation weights For LoRA Adapter Fusion (Complete Datasets)	46
4.11	Client Sample sizes and Aggregation Weights For LoRA Adapter Fusion (75% Training Partition per dataset)	47
5.1	Types of error information during code generation with frozen base model	52
5.2	Error reduction comaparison for FedErroravg and baseline	59
5.3	Error reduction comaparison for FedErroravg and baseline for Test Dataset	60

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Client-Side Hallucination Detection and Dataset Construction	19
3.2	LoRA Fine-Tuning and Federated Aggregation	21
5.1	Hallucination Vs Passed count across dataset	51
5.2	Hallucination count (Baseline Vs FedAvg)	53
5.3	Hallucination count (Baseline Vs FedAvg) on test data	54
5.4	Hallucination count (Baseline Vs FedErrorAvg)	55
5.5	Hallucination count (Baseline Vs FedAvg) on test data	56
5.6	Pass rate comparison among Baseline, FedAvg and FedErrorAvg	57
5.7	Pass rate comparison among Baseline, FedAvg and FedErrorAvg for test dataset	57
5.8	CodeBLEU comparison between Baseline, Fedavg, Federroravg	61

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
APR	Automatic Program Repair
AST	Abstract Syntax Tree
BLEU	Bilingual Evaluation Understudy
CodeBLEU	Code Bilingual Evaluation Understudy
CSV	Comma Separated Values
DS-1000	Data Science 1000
FedAvg	Federated Averaging
FedErrorAvg	Federated Error Average
FedFT	Federated Fine Tuning
FL	Federated Learning
GPU	Graphics Processing Unit
IID	Independent and Identically Distributed
LLM	Large Language Model
LoRA	Low Rank Adaptation
MBPP	Mostly Basic Programming Problems
MLP	Multi-Layer Perceptron
NLP	Natural Language Processing
PEFT	Parameter Efficient Fine Tuning
RLHF	Reinforcement Learning from Human Feedback
SFT	Supervised Fine Tuning